Foundations of Machine Learning and Deep Learning

1. Introduction to Machine Learning

University Master's Degree in
Machine Learning

upna

Universidad Pública de Navarra
Nafarroako Unibertsitate Publikoa

Outline

- What is Machine Learning?
- Notation & Core Concepts
- Learning paradigms
- Classical Models
  - K nearest neighbors
  - Naive Bayes
  - Decision trees
  - Ensemble models
- Model evaluation and selection
- Clustering

Classical Models – Naive Bayes

- **Classification models**

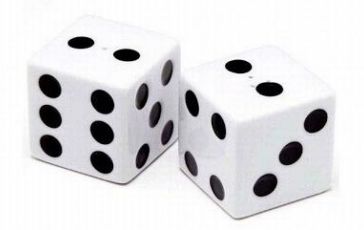
GENERATIVE
Model the distribution of each class, to understand how are all the data of the class

NAIVE BAYES

DISCRIMINATIVE
Model de decisión function to look for the border that separates the classes

Classical Models – Naive Bayes

- **Revisiting probability**
  - Joint probability $$P(A, B) = P(B, A) = P(A \cap B)$$
  - Conditional probability $$P(A \mid B) = \frac{P(A, B)}{P(B)} \quad \text{if } P(B) > 0$$
  - Product rule $$P(A, B) = P(A \mid B)P(B)$$
  - Independence $$P(A, B) = P(A)P(B)$$ $$P(A \mid B) = P(A)$$



Classical Models – Naive Bayes

- Naive Bayes is a probabilistic classifier based on Bayes' Theorem
- Its goal is to estimate the posterior probability for each class $(y\,|\boldsymbol{x})$ and apply the Maximum A Posteriori (MAP) decision rule to select the most probable class

$$\hat{y} = \arg\max_{y} P(y \mid \mathbf{x})$$

Not available from trainig data

Classical Models – Naive Bayes

$$P(y \mid \mathbf{x}) = \frac{P(\mathbf{x} \mid y)P(y)}{P(\mathbf{x})}$$

- $P(y|x)$ is known as the posterior (A posteriori). What we want to calculate
- $P(\mathrm{x}|\mathrm{y})$ is known as the likelihood. If the class is y, how frequent is to see x
- $P(y)$ is known as the prior (A priori). How frequent is the class y
- $P(x)$ is known as the marginal. Probability of observing x

*$P(x)$is constant for all classes y, so it can be dropped for class decision ($\arg\max$).

Classical Models – Naive Bayes

- **Computation**

$$P(y \mid \mathbf{x}) = \frac{P(\mathbf{x} \mid y)P(y)}{P(\mathbf{x})} \Rightarrow \boxed{P(y \mid \mathbf{x}) \propto P(\mathbf{x} \mid y)P(y)}$$

$$\begin{aligned}
P(\mathbf{x} \mid y) &= P(x_1, x_2, \ldots, x_d \mid y) \\
&= P(x_1 \mid y) \cdot P(x_2, \ldots, x_d \mid y, x_1) \\
&= P(x_1 \mid y) \cdot P(x_2 \mid y, x_1) \cdot P(x_3, \ldots, x_d \mid y, x_1, x_2) \\
&\vdots \\
&= P(x_1 \mid y) \cdot P(x_2 \mid y, x_1) \cdot \cdots \cdot P(x_d \mid y, x_1, \ldots, x_{d-1}) \\
&= \prod_{j=1}^{d} P(x_j \mid y, x_1, \ldots, x_{j-1})
\end{aligned}$$

Classical Models – Naive Bayes

- **Computation**

$$P(y \mid \mathbf{x}) \propto P(\mathbf{x} \mid y)P(y)$$

$$P(\mathbf{x} \mid y) = \prod_{j=1}^{d} P(x_j \mid y, x_1, \ldots, x_{j-1})$$

Rare in real life, but works well in practice

Naive Bayes assumption:

- All features are conditionally independent (given the class label)

$$P(x_i \mid x_k, y) = P(x_i \mid y) \quad \text{for all } i \neq k$$

$$\Rightarrow \quad P(\mathbf{x} \mid y) = \prod_{j=1}^{d} P(x_j \mid y) \quad \Rightarrow \quad P(y \mid \mathbf{x}) \propto P(y) \cdot \prod_{j=1}^{d} P(x_j \mid y)$$

Classical Models – Naive Bayes

- **Final classification**
  - Predict the class with highest posterior (MAP)

$$P(y \mid \mathbf{x}) \propto P(y) \cdot \prod_{j=1}^{d} P(x_j \mid y)$$

$$\hat{y} = \arg\max_{y \in \mathcal{Y}} P(y) \cdot \prod_{j=1}^{d} P(x_j \mid y)$$

  - To avoid numerical underflow, use logarithms:

$$\hat{y} = \arg\max_{y \in \mathcal{Y}} \log P(y) + \sum_{j=1}^{d} \log P(x_j \mid y)$$

Relative frecuency of each class in training data

How do we compute $P(x_j \mid y)$?
- Different variants

Classical Models – Naive Bayes

- **How do we compute $P(x_j \mid y)$?**
  - Depends on the assumed distribution of the feature values
    - For each feature and each class, we select a distribution and estimate its parameters
  - We focus on three variants:

Variant	Feature Type	Assumption
Categorical NB	Categorical (0,1,2,…)	Categorical distribution
Multinomial NB	Discrete counts	Multinomial distribution
Gaussian NB	Continuous values	Normal distribution

Classical Models – Naive Bayes

- **Categorical Naive Bayes**
  - Models features with discrete, unordered values
  - Suitable for:
    - Binary features (0/1)
    - Nominal categorical data (e.g., “red”, “green”, “blue” → 0,1,2)
  - Each feature is modeled with a full probability table

$$P(x_j = v \mid y)$$

(for each possible value *v*)

Classical Models – Naive Bayes

- **Categorical Naive Bayes**
  - For each feature x_j and class y, estimate:

$$P(x_j = v \mid y) = \frac{\text{count}(x_j = v, y)}{\text{count}(y)}$$

    - **Numerator**: how many times value *v* of feature x_j appears in class y
    - **Denominator**: total number of training simples with label y

  - Problem: Zero probabilities
    - If a value *v* is never seen in class *y*, the probability is 0 and it kills the product

$$\prod_j P(x_j = v_j \mid y) = 0$$



Laplace smoothing

Classical Models – Naive Bayes

- **Categorical Naive Bayes**
  - Laplace smoothing

$$P(x_j = v \mid y) = \frac{\text{count}(x_j = v, y) + \alpha}{\text{count}(y) + \alpha \cdot K_j}$$

where

- α is the smoothing parameter (typically 1)
- K_j is the number of posible values of feature x_j

- **Categorical Naive Bayes - Example**

Sample	Color	Buy
1	Red	Yes
2	Green	Yes
3	Red	Yes
4	Green	No
5	Red	No
6	Blue	No
7	Red	Yes
8	Green	Yes

Training data



Compute frecuencies

Color	Count in Yes	Count in No
Red	3	1
Green	2	1
Blue	0	1
Total	**5**	**3**

Classical Models – Naive Bayes

- **Categorical Naive Bayes – Example**

  - For class “Yes”:

$$P(\text{Color} = v \mid \text{Yes}) = \frac{\text{count}(v, \text{Yes}) + 1}{5 + 3} = \frac{\text{count} + 1}{8}$$

  - For class “No”:

$$P(\text{Color} = v \mid \text{No}) = \frac{\text{count}(v, \text{No}) + 1}{3 + 3} = \frac{\text{count} + 1}{6}$$

Color	Count in Yes	Count in No
Red	3	1
Green	2	1
Blue	0	1
Total	**5**	**3**

Color	Smoothed Probability (Yes)	Smoothed Probability (No)
Red		
Green		
Blue		

- **Categorical Naive Bayes – Example**
  - **Input**: new sample (color = blue)
    - Use class priors: $P(\text{Yes}) = \frac{5}{8} \quad P(\text{No}) = \frac{3}{8}$
    - Use smoothed probabilities: $P(\text{Color} = 2 \mid \text{Yes}) = \frac{1}{8} = 0.125$

      $P(\text{Color} = 2 \mid \text{No}) = \frac{2}{6} \approx 0.333$
    - Compute posterior scores:

$$\text{Score}(\text{Yes}) = P(\text{Yes}) \cdot P(\text{Color} = 2 \mid \text{Yes}) = \frac{5}{8} \cdot \frac{1}{8} = \frac{5}{64} \approx 0.078$$

$$\text{Score}(\text{No}) = P(\text{No}) \cdot P(\text{Color} = 2 \mid \text{No}) = \frac{3}{8} \cdot \frac{2}{6} = \frac{6}{48} = \frac{1}{8} = \boxed{0.125}$$

Predict: No

Classical Models – Naive Bayes

- **Multinomial Naive Bayes**
  - Models with count-based features
  - Input: vector with **non-negative** integers
    - Ignores features where $x_j = 0$
  - Common in NLP (bag of words, TF-IDF Word counts)

  - Estimates the likelihood of observing feature counts (e.g., word frequencies) given a class

$$P(\mathbf{x} \mid y) \propto \prod_{j=1}^{d} P(x_j \mid y)^{x_j}$$

Classical Models – Naive Bayes

- **Multinomial Naive Bayes**
  - For each class *y* and word *j:*

$$P(x_j \mid y) = \frac{\text{count}(x_j \text{ in class } y) + \alpha}{\sum_k \text{count}(x_k \text{ in class } y) + \alpha \cdot V}$$

where

- α is the smoothing parameter (typically 1)
- V is the number of features (e.g., vocabulary size)

  - **Key difference**:
    - Zero counts are ignored (a word that doesn't appear has no contribution)

Classical Models – Naive Bayes

- **Multinomial Naive Bayes – Example**
  - Classifying a message as Spam or Not-spam
    - Vocabulary: “win”, “free”, “meeting” ($V = 3$)
    - If word appears: 1, else 0
      - For the example, we ignore other words

Word	Spam Count	Not-spam Count
win	10	1
free	5	0
meeting	1	6
Total	**16**	**7**

    - Message to classify: “Let’s schedule the meeting”

Classical Models – Naive Bayes

- **Multinomial Naive Bayes – Example**
  - Spam or Not-spam

Word	Spam Count	Not-spam Count
win	10	1
free	5	0
meeting	1	6
Total	**16**	**7**

  - Message to classify: "*Let's schedule the meeting*" (using $\alpha = 1$)
  - Priors: $$P(\text{Spam}) = \frac{16}{16+7} = \frac{16}{23}, \quad P(\text{Not-spam}) = \frac{7}{23}$$
  - For spam: $$P(\text{meeting} \mid \text{Spam}) = \frac{1+1}{16+3} = \frac{2}{19}$$
  - For not-spam: $$P(\text{meeting} \mid \text{Not-spam}) = \frac{6+1}{7+3} = \frac{7}{10}$$

Classical Models – Naive Bayes

- **Multinomial Naive Bayes – Example**
  - Spam or Not-spam

Word	Spam Count	Not-spam Count
win	10	1
free	5	0
meeting	1	6
Total	**16**	**7**

  - Message to classify: "*Let's schedule the meeting*" (using $\alpha = 1$)

$$\text{Score}(\text{Spam}) = P(\text{Spam}) \cdot P(\text{meeting} \mid \text{Spam}) = \frac{16}{23} \cdot \frac{2}{19} = \frac{32}{437} \approx 0.073$$

$$\text{Score}(\text{Not-spam}) = \frac{7}{23} \cdot \frac{7}{10} = \frac{49}{230} \approx \boxed{0.213}$$

Predict: Not-spam

Classical Models – Naive Bayes

- **Gaussian Naive Bayes**
  - Assumes features to be **continuous** and to follow a normal (Gaussian) distribution

$$x_j \mid y \sim \mathcal{N}(\mu_{j,y}, \sigma^2_{j,y})$$

  - Uses the Gaussian probability density function for likelihoods:

$$P(x_j \mid y) = \frac{1}{\sqrt{2\pi\sigma^2_{j,y}}} \exp\left(-\frac{(x_j - \mu_{j,y})^2}{2\sigma^2_{j,y}}\right)$$

where

- $\mu_{j,y}$ is the mean of feature *j* for class *y*
- $\sigma^2_{j,y}$ is the variance of feature *j* for class *y*

These are the parameters that are learnt

Classical Models – Naive Bayes

- **Gaussian Naive Bayes – Example**
  - Classifying a person as athlete or non-athlete based on height and weight

Sample	Height	Weight	Class
1	172	65	Athlete
2	170	64	Athlete
3	168	66	Athlete
4	158	80	Non-athlete
5	162	78	Non-athlete
6	160	82	Non-athlete

Training data

Classical Models – Naive Bayes

- **Gaussian Naive Bayes – Example**
  - Classifying a person as athlete or non-athlete based on height and weight

Sample	Height	Weight	Class
1	172	65	Athlete
2	170	64	Athlete
3	168	66	Athlete
4	158	80	Non-athlete
5	162	78	Non-athlete
6	160	82	Non-athlete

Priors

$$P(\text{Athlete}) = P(\text{Non-athlete}) = 0.5$$

Athlete

Feature	Mean (μ)	Std (σ)
Height	170	2
Weight	65	1

Non-athlete

Feature	Mean (μ)	Std (σ)
Height	160	2
Weight	80	2

Classical Models – Naive Bayes

- **Gaussian Naive Bayes – Example**
  - Classifying a person as athlete or non-athlete based on height and weight

Priors

$$P(\text{Athlete}) = P(\text{Non-athlete}) = 0.5$$

Athlete

Feature	Mean (μ)	Std (σ)
Height	170	2
Weight	65	1

Non-athlete

Feature	Mean (μ)	Std (σ)
Height	160	2
Weight	80	2

- New sample: height = 169, weight = 66

$$P(x_j \mid y) = \frac{1}{\sqrt{2\pi\sigma_{j,y}^2}} \exp\left(-\frac{(x_j - \mu_{j,y})^2}{2\sigma_{j,y}^2}\right)$$

  - What is the prediction? Do it as an exercise

Classical Models – Naive Bayes

Summary

- Naive Bayes is a simple and fast probabilistic classifier
- It assumes conditional Independence of features given the class
- Despite its simplicity, it performs well in many tasks

- It is a generative model:
  - It models how the data is generated by learning
    - The prior $P(y)$
    - The likelihood $P(\mathbf{x} \mid y)$
  - Uses Bayes' rule to compute $P(y \mid \mathbf{x})$
- Other models learn $P(y \mid \mathbf{x})$ directly

